

Regression splines, local regression methods and
kernel methods

Non-linear models

- ▶ Polynomial regression, e.g., linear regression on X , X^2 and X^3 . Y is a non-linear function of X , albeit not of the regression coefficients.
- ▶ Step functions - piecewise constant regression
- ▶ Regression splines - piecewise polynomials
- ▶ Local regression - regression where data points near (in predictor space) the one that shall be predicted are weighted more than points far away.
- ▶ Smoothing splines - Least squares with a smoothness penalty
- ▶ Generalized additive models - a framework to generalize the methods above to the case with more than one x -variable.

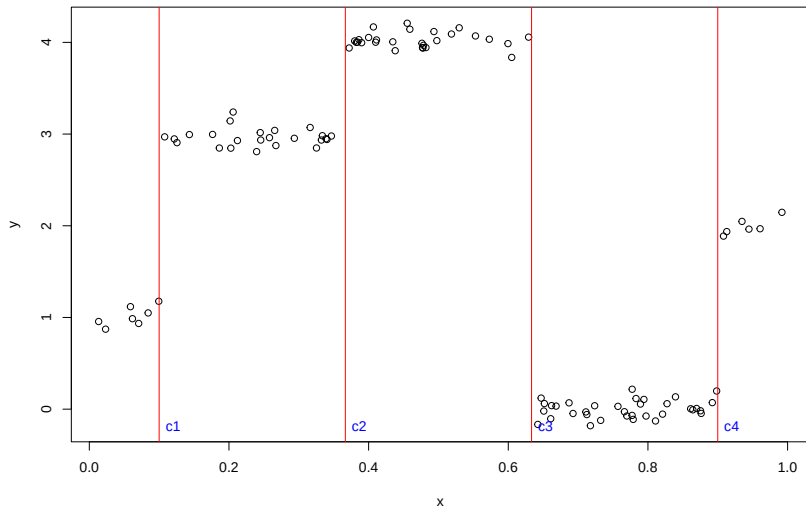
Non-linear models

Step function - piecewise constant regression, e.g.,

$$y = \beta_0 + \beta_1 I(x < c_1) + \beta_2 I(c_1 \leq x < c_2) + \beta_3 I(c_2 \leq x < c_3) \\ + \beta_4 I(c_3 \leq x < c_4) + \beta_5 I(x \geq c_4) + \varepsilon$$

```
# SIMULATE DATA
set.seed(1)
n=100
x=runif(n)
c=seq(0.10,0.90,length=4)
b=c(1,2,3,-1,1)
y=b[1]+b[2]*(x<c[1])+b[3]*((x>= c[1]) & x<c[2])+
  b[4]*((x>= c[2]) & x<c[3])+b[5]*((x>= c[3]) & x<c[4])+0.1*rnorm(n)
plot(x,y)
```

Piecewise regression



Estimating a piecewise constant regression

```
c  
  
## [1] 0.1000000 0.3666667 0.6333333 0.9000000  
x1=(x<c[1])  
head(round(x,3),10)  
  
## [1] 0.266 0.372 0.573 0.908 0.202 0.898 0.945 0.661 0.629 0.062  
head(x1,10)  
  
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
```

Estimating a piecewise constant regression

```
##
## Call:
## lm(formula = y ~ x2 + x3 + x4 + x5)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.18730 -0.05677 -0.01480  0.05337  0.27707
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.01326    0.03524   28.76  <2e-16 ***
## x2TRUE         1.94983    0.04005   48.69  <2e-16 ***
## x3TRUE         3.00998    0.03939   76.41  <2e-16 ***
## x4TRUE        -1.01275    0.03860  -26.24  <2e-16 ***
## x5TRUE         0.97814    0.05186   18.86  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.09322 on 95 degrees of freedom
## Multiple R-squared:  0.9971, Adjusted R-squared:  0.997
## F-statistic: 8157 on 4 and 95 DF, p-value: < 2.2e-16
```

The same as estimating means in each region

```
mean(y[x1])
```

```
## [1] 1.013265
```

```
mean(y[x2])
```

```
## [1] 2.96309
```

```
mean(y[x3])
```

```
## [1] 4.023242
```

```
b=coef(reg)
```

```
round(b[1]+b[-1],6)
```

```
##      x2TRUE      x3TRUE      x4TRUE      x5TRUE
```

```
## 2.963090 4.023242 0.000514 1.991408
```

Example with Wage data

```
require(ISLR)
age=Wage$age
table(cut(age,4))
```

(17.9,33.5]	(33.5,49]	(49,64.5]	(64.5,80.1]
750	1399	779	72

```
fit=lm(wage~cut(age,4),data=Wage)
coef(summary(fit))
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	94.158392	1.476069	63.789970	0.000000e+00
cut(age, 4)(33.5,49]	24.053491	1.829431	13.148074	1.982315e-38
cut(age, 4)(49,64.5]	23.664559	2.067958	11.443444	1.040750e-29
cut(age, 4)(64.5,80.1]	7.640592	4.987424	1.531972	1.256350e-01

Basis functions

- ▶ Transform x to other quantities $b_1(x), b_2(x), \dots, b_K(x)$
- ▶ Example 1: $b_1(x) = x, b_2(x) = x^2, b_3(x) = x^3$ Use linear regression of y on $b_1(x), b_2(x)$ and $b_3(x)$

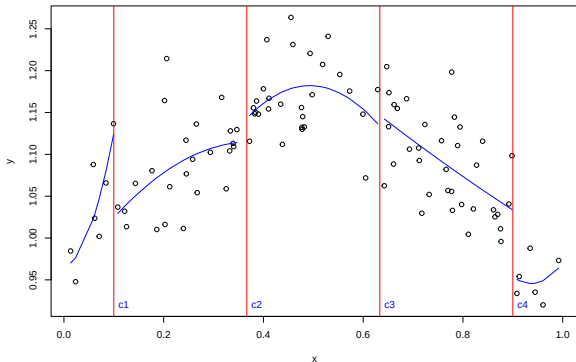
$$y = \beta_0 + \beta_1 b_1(x) + \dots + \beta_K b_K(x) + \varepsilon$$

- ▶ Example 2: $b_k(x) = I(c_k \leq x < c_{k+1})$
- ▶ Example 3: y is a periodic function of x . Then $b_k(x) = \sin(\alpha_k x)$ are good basis functions.

Regression splines

Piecewise polynomials

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon$$



- Different polynomials in each segment.
- Not connected at the *knots* $\xi_1, \xi_2, \dots, \xi_K$ (c instead of ξ in the figure).

Regression splines

- ▶ Knot the pieces together.
- ▶ Constraints:
 - ▶ Curve must be continuous
 - ▶ First two derivatives continuous, i.e. a smooth curve.

Cubic spline basis

$$\begin{aligned}(b_0(x), b_1(x), b_2(x), b_3(x), b_4(x), \dots, b_{K+3}(x)) \\ = (1, x, x^2, x^3, h(x, \xi_1), \dots, h(x, \xi_K))\end{aligned}$$

and their first two derivatives are continuous.

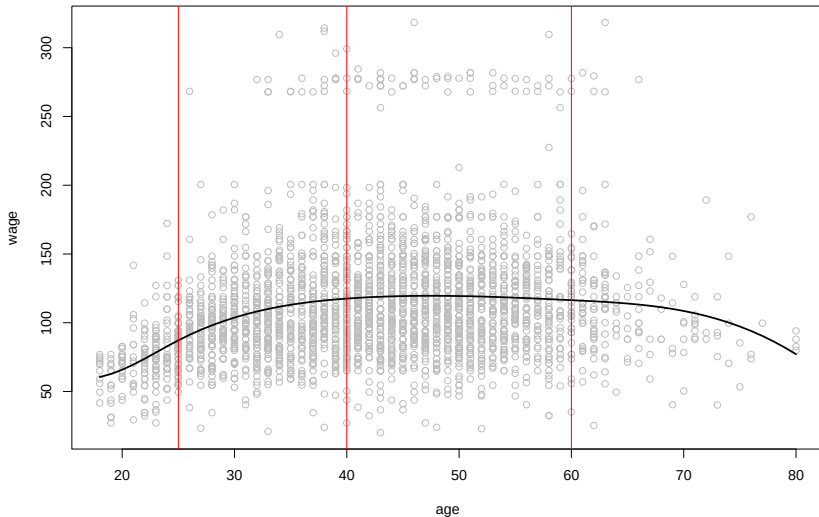
$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$$

$$y = \beta_0 + \beta_1 b_1(x) + \dots + \beta_{K+3} b_{K+3}(x) + \varepsilon$$

Cross-validation to determine number and placement of knots.

Example with Wage data using the splines library

```
fit=lm(wage~bs(age,knots=c(25,40,60)),data=Wage)
```



Kernel estimators



$$y = f(x) + \varepsilon$$

- ▶ K nearest neighbors(KNN)

$$\hat{f}(x_0) = \frac{1}{K} \sum_{k=1}^K y_{(k)}$$

where $y_{(1)}, \dots, y_{(K)}$ are the y 's corresponding to the K x 's closest to x_0

- ▶ Nadaraya-Watson estimator

$$\hat{f}(x_0) = \sum_{i=1}^n w_i y_i$$

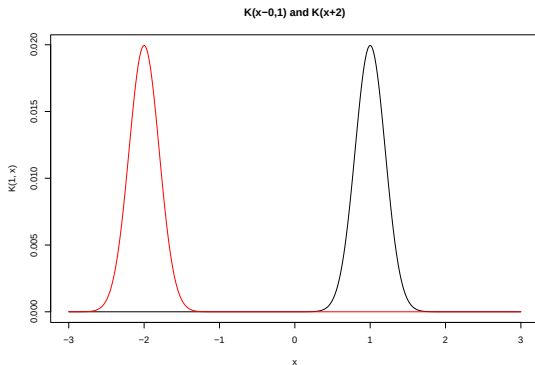
where $\sum_{i=1}^n w_i = 1$ and $w_i = K(x_i - x_0)$

- ▶ K is a so called kernel.

Example of a kernel

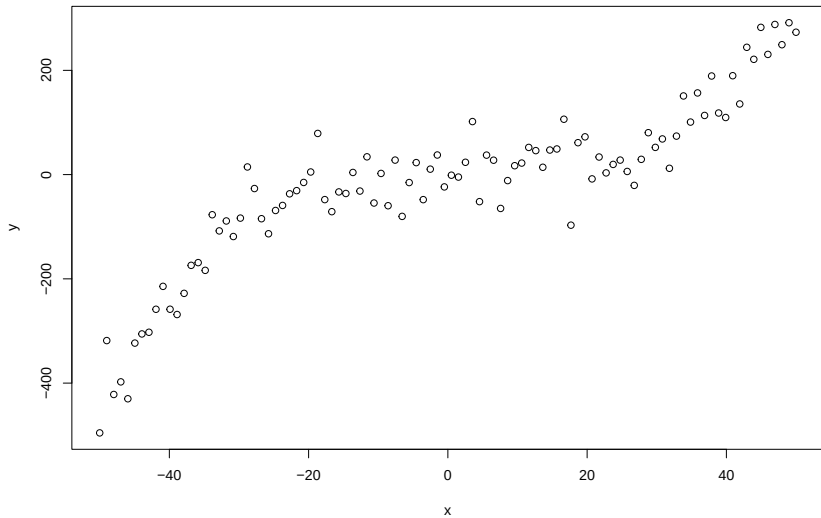
$$K(x_i - x_0) = \frac{\phi\left(\frac{x_i - x_0}{h}\right)}{\sum_{i=1}^n \phi\left(\frac{x - x_0}{h}\right)}$$

where ϕ is the standard normal density.



If we want to predict y based for an observation with $x = -2$ we should give high weights for observations with x close to -2 .

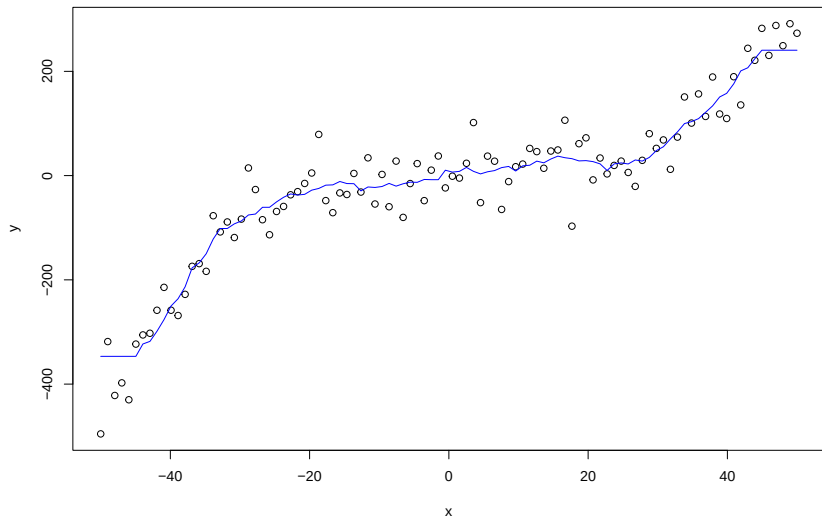
Implementation in R



K nearest neighbor

```
knn=function(x0,x,y,K=sqrt(length(x)))  
{  
  dist=abs(x-x0)  
  odist=order(dist)  
  y0=mean(y[odist[1:K]])  
  return(y0)  
}  
ypred=apply(as.matrix(x),1,knn,x=x,y=y)  
plot(x,y)  
lines(x,ypred,col="blue")
```

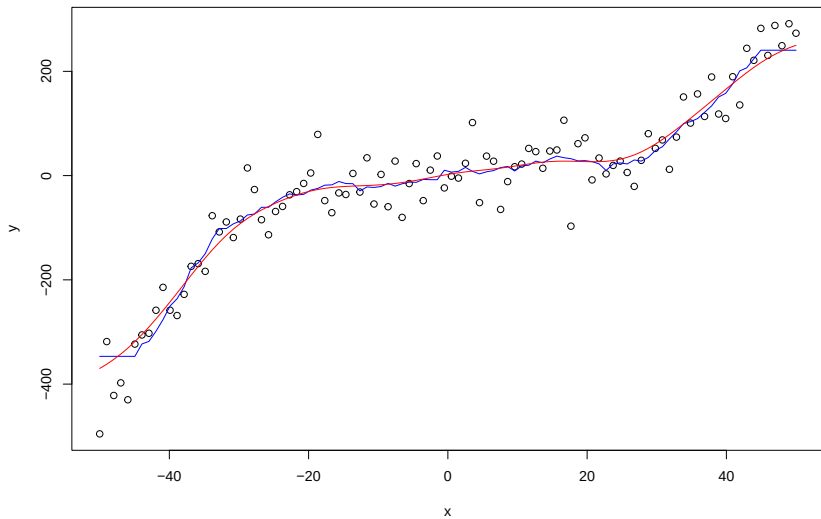
K nearest neighbor



Implementing Nadaraya-Watson

```
K=function(x0,x,h=0.2) dnorm((x-x0)/h)/sum(dnorm((x-x0)/h))
nw=function(x0,x,y,h=0.2)
{
  w=K(x0,x,h=h)
  ypred=sum(w*y)
  return(ypred)
}
ypred=apply(as.matrix(x),1,knn,x=x,y=y)
plot(x,y)
lines(x,ypred,col="blue")
yprednw=apply(as.matrix(x),1,nw,x=x,y=y,h=5)
lines(x,yprednw,col="red")
```

Implementing Nadaraya-Watson



apply can be used instead of a loop

Example: Compute the mean of each column in a matrix

```
X
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    3    5    7
## [2,]    2    4    6    8
```

```
for(i in 1:4) print(mean(X[,i]))
```

```
## [1] 1.5
## [1] 3.5
## [1] 5.5
## [1] 7.5
```

apply can be used instead of a loop

Example: Compute the mean of each column in a matrix

```
X
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    3    5    7
## [2,]    2    4    6    8
```

```
apply(X,2,mean)
```

```
## [1] 1.5 3.5 5.5 7.5
```

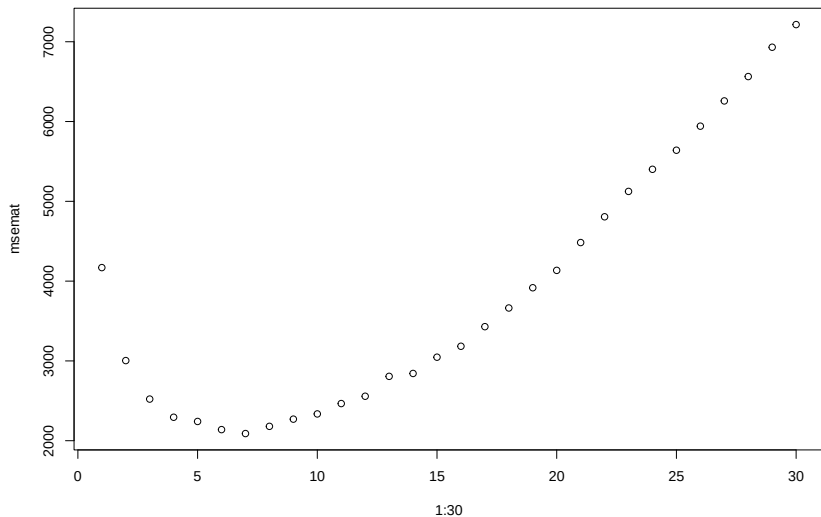
```
apply(X,1,mean)
```

```
## [1] 4 5
```

Determining K by LOOCV

```
mse=function(err) mean(err^2)
n=length(x)
msemat=matrix(0,30,1)
for(k in 1:30)
{
  ypred=matrix(0,n,1)
  for(i in 1:n)
  {
    trainx=x[-i]
    trainy=y[-i]
    testx=x[i]
    ypred[i]=knn(testx,trainx,trainy,K=k)
  }
  err=y-ypred
  msemat[k]=mse(err)
}
plot(k,msemat)
```

Determining K by LOOCV

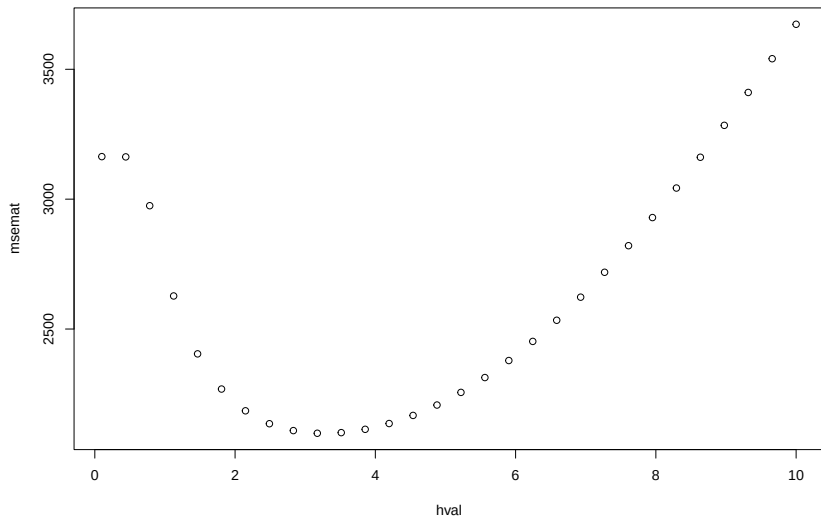


Choose $K = 7$

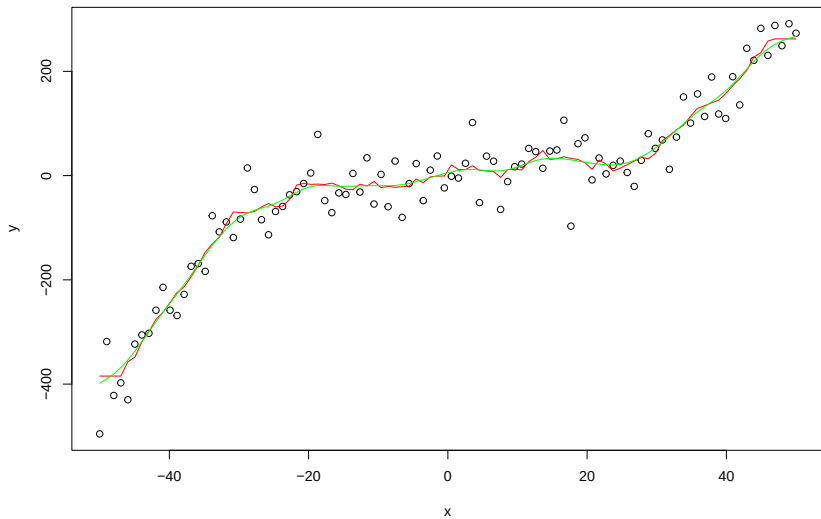
Determining h in Nadaraya-Watson by LOOCV

```
n=length(x)
hval=seq(0.1,10,length=30)
msemat=matrix(0,30,1)
for(k in 1:30)
{
  ypred=matrix(0,n,1)
  for(i in 1:n)
  {
    trainx=x[-i]
    trainy=y[-i]
    testx=x[i]
    ypred[i]=nw(testx,trainx,trainy,h=hval[k])
  }
  err=y-ypred
  msemat[k]=mse(err)
}
```

Determining h in Nadaraya-Watson by LOOCV



Optimal KNN and optimal Nadaraya-Watson



Local linear regression at $x = x_0$

- ▶ Same principle as in the kernel methods: Weight observations with x -values close to x_0 highly.
- ▶ Solve the minimization problem

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n w_i (y_i - \beta_0 - \beta_1 x_i)^2$$

where, e.g., $w_i = \phi\left(\frac{x_i - x_0}{h}\right)$. h is called the bandwidth and can be found by cross-validation.



$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$$